

Appendix for DDC

A HIERARCHICAL BITMAP INDEXES

Hierarchical bitmap indexes summarize lower-level blocks at higher levels and avoid storing empty (or completely full) blocks. For example, HBI [10] organizes each indexed set as a tree of fixed-size bitmap blocks, omits empty blocks, and stores its non-root blocks in a linked list. TEB [6] instead builds and prunes a binary tree, then serializes its topology and leaf labels into packed bit sequences using level-order encoding [4].

Although these designs achieve high compression and support efficient membership queries, **they are inefficient for logical bitwise operations** because traversing a compressed bitvector requires navigating an explicit or implicit tree. For example, HBI follows linked nodes, while TEB traverses its encoded tree using *skip()*. These data-dependent, variable-length access paths are poorly suited to merging two bitvectors (§3). However, DBMS bitmap indexing requires efficient merging of bitvectors from different attributes. To meet this need, **DDC uses contiguous, positionally aligned control layers rather than a tree topology**, enabling branch-free, fixed-length SIMD kernels for logical bitwise operations. Across AND, OR, NOT, and COMP, DDC is $9\times$ – $1,877\times$ faster than HBI and $21\times$ – $2,282\times$ faster than TEB (R3.W2).

Logical Bitwise Operations. We evaluate COMP on equality bitmaps from the ten real-world IMDb attributes in Table 5.

End-to-End Benefits in DuckDB. We evaluate eight JOB queries over IMDb using native DuckDB and bitmap-indexed DuckDB variants. The bitmap-indexed variants rewrite fact–dimension joins as predicates on fact-table foreign keys and evaluate those predicates using the corresponding bitmap indexes (§5.5). Figure 2 reports end-to-end latency for native DuckDB and the DDC and CRoaring variants. We omit WAH and EWAH because they perform similarly to CRoaring. Across all eight queries, DDC delivers $1.6\times$ – $8.2\times$ speedups over native DuckDB. CRoaring narrows the gap on queries involving sparse or highly clustered bitvectors, but its container-dependent execution remains sensitive to input patterns (R3.W1). In contrast, DDC’s control–data separation enables branch-free, fixed-length SIMD merges, yielding $1.1\times$ – $1.5\times$ speedups over CRoaring.

B DATA DISTRIBUTION

We have expanded our evaluation to cover (1) columns with Zipfian (long-tail) distributions, (2) clustered and fully sorted columns, (3) varying column cardinalities, (4) nonuniform real-world datasets (Berkeley Earth and IMDb), and (5) an uncompressed bitset baseline for TPC-H.

Zipfian (Long-Tailed) Distributions. Frequent values produce dense bitvectors, whereas rare values produce sparse ones. We evaluate all schemes by varying the skew parameter θ of the finite Zipf distribution $P(X = k) = k^{-\theta} / \sum_{j=1}^{100} j^{-\theta}$, where $k \in \{1, \dots, 100\}$ denotes the value rank. At $\theta = 0$, the distribution is uniform. At $\theta = 2$, the largest skew evaluated, the two most frequent values together account for 76.5%. We make the following observations.

DDC is Smallest Across Evaluated Skews. Table 1 compares DDC with the baselines (Concise follows WAH and is omitted for brevity). All compressed indexes shrink monotonically as θ increases, because a

higher θ makes more tail bitvectors sparse, yielding more opportunities for compression [2]. Across all θ , DDC is the smallest, while CRoaring, the closest baseline, requires 2–8% more space.

Table 1: Bitmap-index sizes under varying Zipfian skew (100M rows, cardinality 100; MB). DDC produces the smallest index at every evaluated skew.

θ	DDC	WAH	Inc.	EWAH	Inc.	CRoaring	Inc.	Bitset	Inc.
0	198	598	+2.0×	905	+3.6×	201	+2%	1250	+5.3×
0.5	189	553	+1.9×	827	+3.4×	201	+6%	1250	+5.6×
1.0	156	397	+1.6×	595	+2.8×	168	+8%	1250	+7.0×
1.5	105	229	+1.2×	335	+2.2×	111	+5%	1250	+10.9×
2.0	69	130	+87%	180	+1.6×	72	+4%	1250	+17.0×

DDC is Fast and Robust under Skew. Zipfian columns yield bitvectors with heterogeneous bit densities, so logical operations must handle operand bitvectors with different densities. Figure 3 quantifies this effect by varying both operand densities and measuring OR latency (Other bitwise operations show similar trends and are omitted for brevity). At some density pairs, CRoaring incurs array-bitset container transitions, while WAH, EWAH, and Concise process RLE streams with data-dependent branches (§3). In contrast, DDC separates control from data and sustains SIMD execution across the full density grid. As a result, DDC is up to $23.2\times$, $16.0\times$, $6.4\times$, and $43.0\times$ faster than WAH, EWAH, CRoaring, and Concise, respectively.

Clustered and Fully Sorted Columns. We evaluate DDC and the baselines on clustered columns. Following Wu et al. [14], we define a bitvector’s *clustering factor* CF_1 as its average 1-run length, h/r , where h is the number of set bits and r the number of 1-runs. For a bitmap index, $CF_1 = \sum_i h_i / \sum_i r_i$ across all bitvectors i . A larger CF_1 means fewer, longer 1-runs, which favors RLE-based schemes. We make the following observations.

Table 2: Bitmap sizes as the clustering factor CF_1 varies (100M rows, cardinality 100; MB). DDC is smallest for $CF_1 \leq 4$. For heavily clustered attributes, CRoaring and WAH are smaller.

CF_1	DDC	WAH	Inc.	EWAH	Inc.	CRoaring	Inc.	Bitset	Inc.
1	197.6	598.4	+2.0×	904.8	+3.6×	201.2	+2%	1250	+5.3×
4	80.1	195.5	+1.4×	352.4	+3.4×	101.5	+27%	1250	+14.6×
8	53.6	107.7	+1.0×	196.0	+2.7×	51.6	−4%	1250	+22.3×
16	38.1	60.9	+60%	107.9	+1.8×	26.6	−30%	1250	+31.9×
64	20.0	20.2	+1%	35.4	+77%	7.80	−61%	1250	+61.4×
256	11.4	5.81	−49%	11.1	−2%	3.00	−74%	1250	+108.9×
1K	5.14	1.51	−71%	3.01	−41%	1.14	−78%	1250	+242.1×
2K	3.02	0.763	−75%	1.53	−49%	0.641	−79%	1250	+412.4×
4K	1.68	0.385	−77%	0.773	−54%	0.349	−79%	1250	+743.3×

DDC is Smallest Except for Heavily Clustered Columns. As shown in Table 2, DDC is smallest for uniform or slightly clustered columns ($CF_1 \leq 4$). As CF_1 grows, WAH, EWAH, and CRoaring shrink faster because they apply RLE to both 0-runs and 1-runs, whereas DDC compresses only 0-runs and stores 1-runs verbatim at L1.

DDC is Fastest Except for Heavily Clustered Columns. We measure bitwise throughput while varying CF_1 on 100M-row columns with

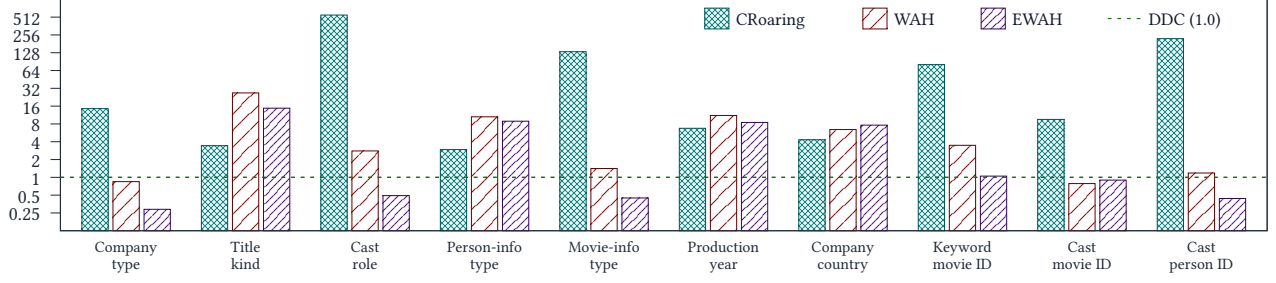


Figure 1: COMP latency on equality bitmaps from ten real-world IMDb attributes, normalized to DDC (lower is better). Each bar reports a baseline’s latency divided by DDC’s latency; the dashed line marks DDC at 1.0, and bars below the line favor the baseline. Across the ten attributes, the geometric-mean latency ratios of CRoaring, WAH, and EWAH to DDC are 23.6, 3.4, and 1.7, respectively. The CRoaring bars report its native COMP path, which may perform internal array–bitset container conversions but excludes any subsequent conversion required to deliver a bitvector to the DBMS.

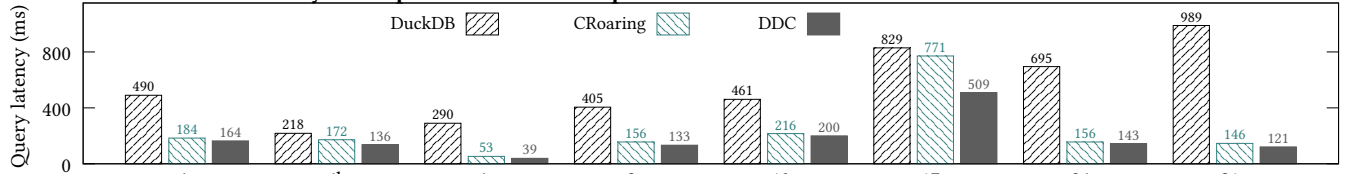


Figure 2: JOB queries latency over the real-world IMDb dataset for the native DuckDB and bitmaps compressed using different schemes. DDC has the lowest latency and delivers geometric-mean speedups of 3.4× over native DuckDB and 1.2× over CRoaring.

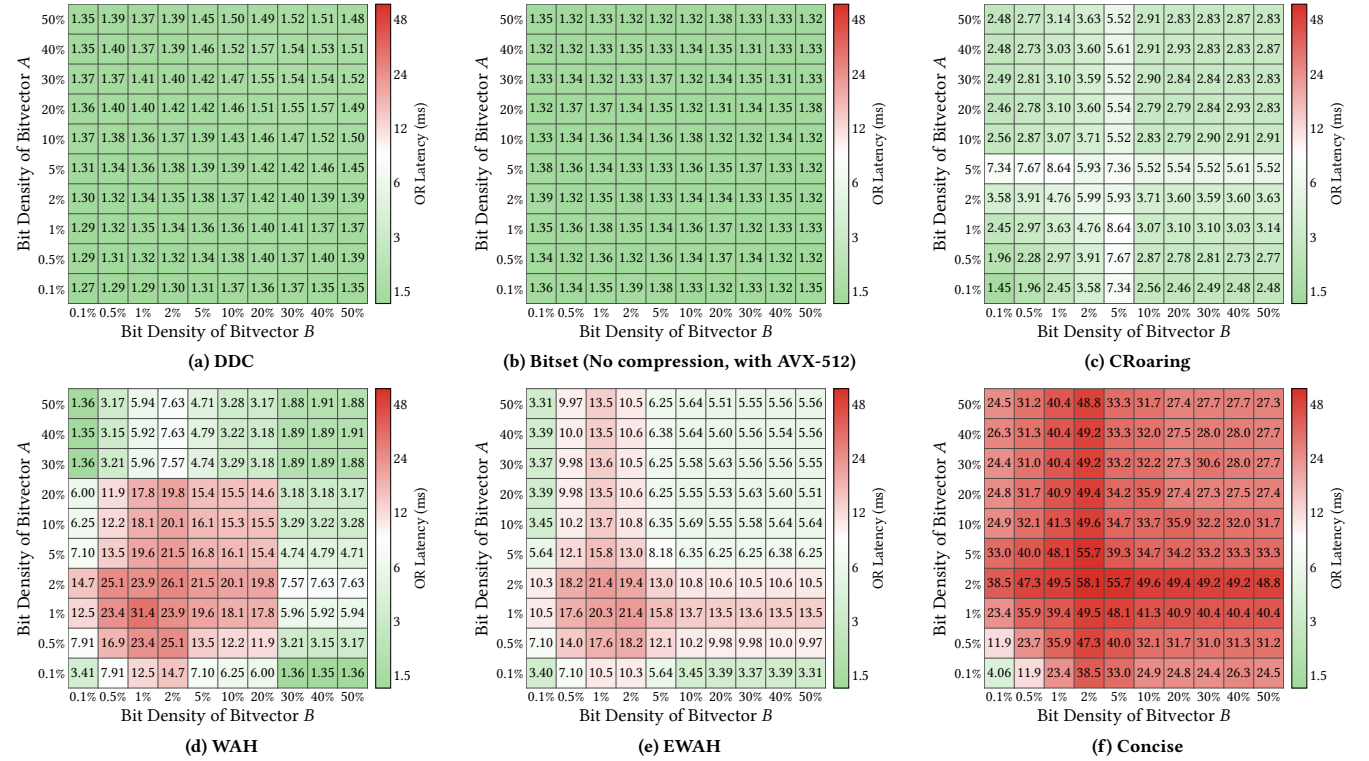


Figure 3: Bitwise OR latency across operand densities $d_A, d_B \in [0.1\%, 50\%]$ for 100M-row Zipfian attributes. DDC (a) approaches the performance of the uncompressed, AVX-512-accelerated Bitset baseline (b) while using substantially less space. The other compressed indexes (c)–(f) exhibit density-dependent slowdowns.

cardinality 100. Figure 4 shows that DDC maintains stable throughput across CF_1 values due to its control-data separation (§4.1). DDC

is fastest on uniform ($CF_1 = 1$) and weakly clustered ($CF_1 < 16$) columns, which represent most attributes in real workloads (R4.W1).

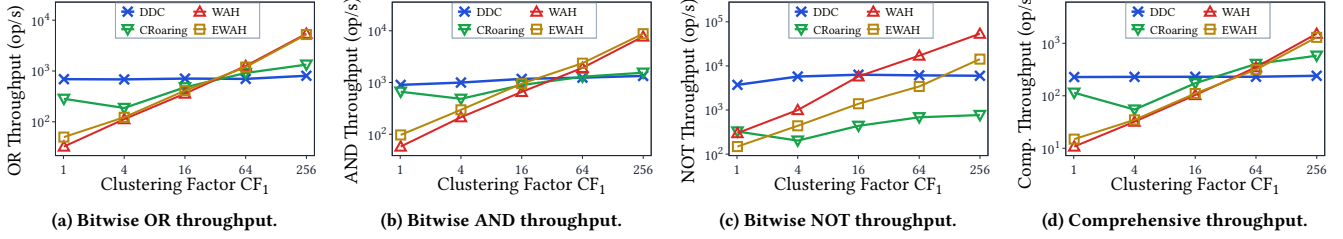


Figure 4: Bitwise throughput (log scale) versus clustering factor CF_1 for OR, AND, NOT, and COMP (100M rows with cardinality 100). DDC maintains stable throughput across all CF_1 values and outperforms all baselines when $CF_1 \leq 16$.

As CF_1 increases, WAH and CRoaring become faster and eventually outperform DDC. This is mainly because WAH and EWAH also encode long 1-runs as fill words, while CRoaring uses run containers, allowing them to skip these runs during bitvector merges.

Discussion. Our evaluation shows that WAH and CRoaring outperform DDC on heavily clustered columns because they explicitly encode 1-runs. **DDC nonetheless retains this asymmetric design for the following reasons.**

- Each table has a single physical row order, so most columns are uniform or weakly clustered, as our real-world results confirm (R4.W1). Although row reordering can benefit [9], existing methods [8, 11] assume an *offline, read-only* setting and do not support the mutable workloads targeted by DDC.
- DDC’s absolute size and latency penalties on the highly clustered columns are small. As CF_1 increases, all schemes shrink (Table 2) and generally become faster (Figure 4). For example, although WAH is smaller than DDC at $CF_1 = 256$, its size advantage is only $11.4 - 5.81 = 5.59\text{MB}$; at $CF_1 = 1$, DDC’s advantage over WAH is $598.4 - 197.6 = 400.8\text{MB}$. Similarly, although WAH is faster than DDC at $CF_1 = 256$, its latency advantage is only $4 - 0.7 = 3.3\text{ms}$; at $CF_1 = 1$, DDC’s advantage over WAH is $93 - 4 = 89\text{ms}$.

Low-to-High Cardinality. Cardinality does not affect our conclusion. Higher cardinality produces sparser bitvectors, allowing DDC and all baselines to exploit this sparsity for compression and skipping during bitwise operations, preserving their relative performance trends. This result aligns with prior findings [2, 13]. Our evaluation further confirms it on uniformly distributed TPC-H attributes with cardinalities of 2 to 15M (§5.2) and real-world IMDb attributes with cardinalities of 2 to 4.1M (see R4.W1).

C HIERARCHICAL BITMAP INDEXES

Our investigation found that *existing hierarchical bitmap designs rely on tree structures* either explicitly as linked nodes [10] or implicitly as succinct bit sequences of pruned trees [6]. For example, the mentioned HBI [10] organizes each indexed set into a hierarchy of fixed-size bitmap nodes, omits empty nodes, and logically organizes the bitmap nodes as a tree while physically storing the non-root nodes in a linked list. TEB [6] instead builds and prunes a binary tree, then serializes its topology and leaf labels into packed bit sequences using a level-order encoding [4]. We evaluate HBI and TEB on our testbed and make the following observations.

DDC is Smallest at Moderate Densities. Table 3 depicts three density regimes. (1) At high density (40%–50%), bitvectors are barely compressible, so TEB stays at the packed 12.5MB label size while DDC’s

control metadata requires $\sim 2\%$ more storage space. (2) At moderate density (5%–20%), DDC omits 0-runs at byte granularity and recursively compresses its control flags, making it 5%–15% smaller than TEB. (3) At low density ($< 5\%$), TEB’s variable-depth tree compresses both 1- and 0-runs at a finer granularity (bits), making it smaller than DDC but only by 3%–17%. HBI remains larger than DDC because it always uses one 32-bit mask per non-empty node.

Table 3: Bitvector sizes across schemes, including HBI and TEB (100M bits; MB). DDC is smallest at densities of 5%–20%. At the other densities, DDC is only 3%–17% larger than TEB.

Density	DDC	HBI	Inc.	TEB	Inc.	Bitset	Inc.
50%	12.7	12.9	+2%	12.5	−2%	12.5	−2%
40%	12.7	12.9	+2%	12.5	−2%	12.5	−2%
20%	11.9	12.9	+8%	12.5	+5%	12.5	+5%
10%	8.8	12.5	+42%	10.1	+15%	12.5	+42%
5%	5.9	10.5	+78%	6.5	+10%	12.5	+1.1×
2%	3.3	6.4	+94%	3.2	−3%	12.5	+2.8×
1%	2.0	3.8	+90%	1.8	−10%	12.5	+5.3×
0.5%	1.2	2.3	+92%	1.0	−17%	12.5	+9.4×

DDC Provides Efficient Bitwise Operations. Although existing hierarchical designs achieve high compression ratios and support efficient membership queries (e.g., HBI tests a small set of element positions for containment, and TEB performs a point lookup at a single bit position), merging these bitvectors requires tree traversal. For bitwise operations, HBI traverses its explicitly linked nodes, whereas TEB navigates its implicitly encoded tree. These traversal paths are data-dependent and scalar-based (§3). In contrast, **DDC realizes hierarchy as positionally aligned control layers rather than as a tree topology**, enabling branch-free bitwise operation kernels (§4.1). Overall, DDC is 12×–44×, 10×–38×, 39×–1,877×, and 9×–41× faster than HBI on AND, OR, NOT, and COMP, respectively. Against TEB, the corresponding speedups are 34×–2,282×, 27×–2,225×, 540×–1,753×, and 21×–881× (Figure 5).

D DDC’S SCAN BENEFITS DRILL DOWN

We evaluate scans from three complementary angles: (1) a latency breakdown of scan-intensive queries, (2) an evaluation of scale-up behavior, and (3) an analysis of the key factors affecting bitmap indexing for scans.

DDC’s Benefits Drill Down. We first perform a performance breakdown of both DDC and DuckDB on scan intensive queries. Figure 6 shows the evaluation results for the classic TPC-H Q6 (SF = 10 and $\sim 2\%$ selectivity). DuckDB spends 291ms scanning *four* referenced LINEITEM columns and evaluating the predicates. In

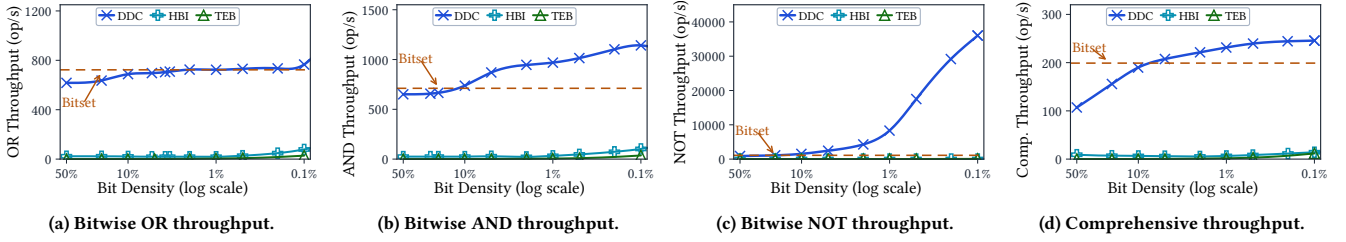


Figure 5: Bitwise throughput across bit densities for 100M-bit inputs, including HBI and TEB. DDC outperforms the baselines across densities. For OR, DDC is $10\times$ – $38\times$ faster than HBI and $27\times$ – $2,225\times$ faster than TEB.

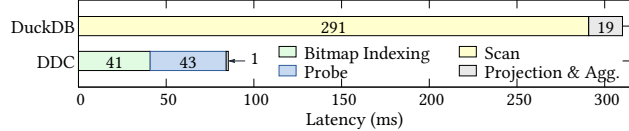


Figure 6: For the scan-intensive TPC-H Q6 (SF = 10), DuckDB scans four columns and then evaluates predicates. In contrast, DDC evaluates predicates via bitmap indexing and then probes the two measure columns, achieving a $3.6\times$ speedup.

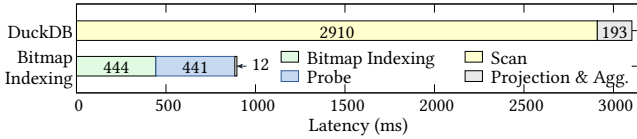


Figure 7: Latency breakdown of the scan-intensive TPC-H Q6 for SF = 100. Compared with SF = 10 (Figure 6), the plotted phase latencies increase by $\sim 10\times$, closely tracking the increase in data size.

contrast, DDC evaluates the predicates using bitmap indexes on $l_shipdate$, $l_discount$, and $l_quantity$ in 41ms, then probes only $l_extendedprice$ and $l_discount$ in 43ms. This reduces end-to-end latency from 311ms to 87ms, a $3.6\times$ speedup. Overall, **DDC accelerates scans by (1) reducing the number of measure columns scanned/probed, and (2) probing only the qualifying column values using indexing results.**

Query Latency Scales Linearly. We repeat the Q6 breakdown at SF = 100, with the database and indexes resident in our testbed’s 192GB memory. Our evaluation (Figure 7) shows that query latency increases by $\sim 10\times$ for both DuckDB and DDC over SF = 10 (Figure 6). We leave cold-cache, disk-resident execution to future work.

Key Factors for Probing vs. Scanning. Finally, we compare probing and scanning the two measure columns as selectivity varies and make two observations. (1) DDC outperforms DuckDB at low selectivity, but its advantage diminishes as selectivity increases (Figure 8). This selectivity-dependent crossover is consistent with prior work on access-path selection [5]. (2) For bitvectors with uniformly distributed set bits, DuckDB and DDC access different numbers of row groups only at extremely low selectivities not encountered in the evaluated TPC-H, IMDb, and Berkeley Earth workloads. For clustered data, DuckDB can also skip nonqualifying row groups using lightweight zone maps [12], so both systems access the same number of row groups. Our evaluation instead shows that **the crossover to DDC’s increasing cost of probing, materializing, and aggregating more qualifying tuples within the accessed row groups at higher selectivities.**

Discussion. Overall, DDC accelerates scans by evaluating predicates with bitmap indexes and probing only qualifying values from the measure columns. Block/page skipping also reduces scan cost, but it applies equally to DDC and DuckDB. Instead, selectivity drives the performance crossover by controlling the number of qualifying values that DDC probes, materializes, and aggregates.

E ABLATION STUDY

We perform an ablation study, starting from *Scalar+No-Skipping* (baseline) to *SIMD+Skipping* (the complete DDC) by enabling each component independently. Because the other operations show similar trends, we only report OR (Figure 9) for brevity. We make the following observations.

SIMD Provides a Notable, Density-Robust Speedup. As bit density varies, *SIMD+No-skipping* sustains $\sim 480\text{op/s}$ across the evaluated densities. This gain comes from DDC’s control-data separation, which replaces data-dependent, word-by-word scalar merging with fixed-length AVX-512 kernels (§4.1). In contrast, both scalar variants slow down as bit density decreases from 50% to 10%. Intel Pin measurements confirm this effect: the branch-misprediction rates of the two scalar variants rise from 3.7%–5.4% at 50% density to 24% at 10% density, whereas both SIMD variants remain below 0.9%.

Skipping Is More Effective at Lower Densities. At low densities, bitvectors contain long runs of 0s, increasing the benefit of skipping. For example, *Scalar+Skipping* is $9\times$ faster than *Scalar+No-Skipping* at 0.02% density, and its speedup grows to $17\times$ at 0.002%.

Conclusion. Overall, as the reviewer suggested, SIMD execution and hierarchical skipping are the two primary sources of DDC’s performance gains and provide complementary benefits. SIMD eliminates data-dependent scalar merging, while skipping accelerates DDC on sparse bitvectors by skipping fill regions.

F REAL-WORLD WORKLOADS

We add two real-world datasets with naturally skewed value distributions and clustered bit patterns: the Berkeley Earth temperature dataset [3] and the IMDb dataset used by the Join Order Benchmark (JOB) [7].

We report cardinality, bit density, and the bitmap clustering factor CF_1 (defined in R3.W1) for each dataset. Real-world indexed columns rarely follow an exact Zipf distribution, so we characterize their skewness using a measured statistic G that summarizes value-frequency imbalance. Specifically, for a column with C distinct values, let n_i denote the number of rows containing the i -th value

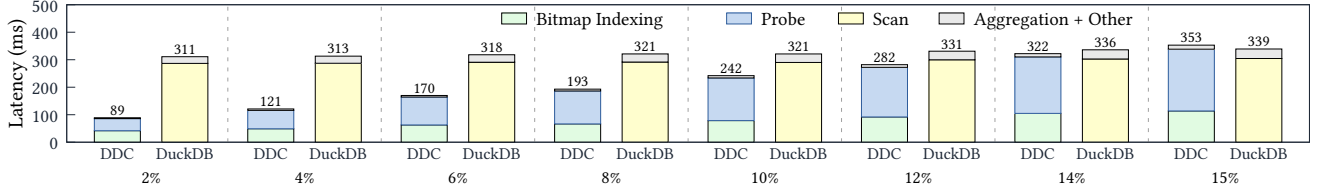


Figure 8: For TPC-H Q6 (SF = 10), DuckDB’s latency remains as query selectivity increases. In contrast, DDC’s latency increases from 89ms to 353ms as it materializes and probes more qualifying rows, yielding a crossover between 14% and 14.5%.

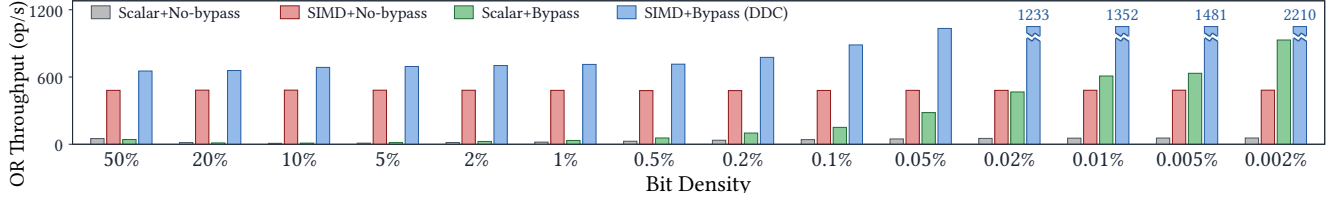


Figure 9: Ablation study isolating the contributions of SIMD execution and hierarchical skipping to DDC’s OR throughput. SIMD improves throughput across bit densities, while skipping benefits on sparse bitvectors by skipping fill regions.

Table 4: Characteristics and sizes of bitmaps on five Berkeley Earth columns (21M rows; MB).

Bitmap	Card.	Density	CF_1	G	DDC	WAH	EWAH	CRoaring
<i>month</i>	12	8.33%	1.0	0.003	26.0	33.8	32.8	32.8
<i>qflag</i>	17	5.88%	10.3	0.751	2.6	3.4	4.3	5.7
<i>decade</i>	33	3.03%	87.1	0.696	2.7	3.7	6.6	1.1
<i>temp_bin</i>	232	0.43%	1.1	0.695	36.4	97.5	128.2	43.9
<i>year</i>	325	0.31%	11.1	0.702	11.4	18.3	33.8	8.5
Total	—	—	—	—	78.9	156.7	205.5	92.0

and $N = \sum_{i=1}^C n_i$ the total number of rows. We define G as

$$G = \frac{\sum_{i=1}^C \sum_{j=1}^C |n_i - n_j|}{2CN}$$

The measured G increases monotonically with the Zipfian parameter θ , starting at 0 for a uniform distribution and approaching 1 as value frequencies become more uneven. For example, the Zipfian columns evaluated in R3.W1 with θ values of 0, 0.5, 1.0, 1.5, and 2.0 have G values of 0.0005, 0.2876, 0.6245, 0.8560, and 0.9465, respectively. We therefore use each attribute’s measured G to represent its Zipfian skew level.

Berkeley Earth provides a high-resolution land and ocean temperature archive [3] with 21M rows. We build equality bitmap indexes on *month*, *year*, *decade*, temperature grouped into 0.5°C bins (*temp_bin*), and a composed key taken directly from each record’s first quality/source code (*qflag*). Together, these five indexes span cardinalities of 12–325 and contain 619 bitvectors, with mean densities of 0.31%–8.33%, CF_1 values of 1.0–87.1, and G values of 0.003–0.751 (Table 4). These include nearly uniform (*month*), strongly clustered (*decade*), and highly skewed (*qflag*) columns. We make the following observations.

DDC Has the Smallest Aggregate Footprint. As shown in Table 4, DDC occupies 78.9MB in total, compared with 92.0MB for CRoaring, the next-smallest scheme. Column skewness and cardinality alone do not determine compressed size. For example, both the nearly uniform column *month* ($G = 0.003$) and the skewed *temp_bin* column ($G = 0.695$) favor DDC. In contrast, clustering favors both WAH and CRoaring. For example, for *decade*, $CF_1 = 87.1$ indicates long 1-runs

that reduce CRoaring’s footprint to 1.1MB, 60% below DDC’s 2.7MB. These results align with DDC’s robustness analysis in R3.W1.

DDC Provides Fast Logical Bitwise Operations. We report OR and COMP latencies across schemes, normalized to DDC (Figure 10). We make three observations. (1) DDC is fastest in all evaluated bitwise operations. The geometric-mean latency ratios of CRoaring, EWAH, and WAH relative to DDC are 2.18×, 5.11×, and 6.83×, respectively. (2) DDC remains robust across column skew and cardinality. E.g., it outperforms all three baselines across five columns spanning G values of 0.003–0.751 and cardinalities of 12–325. (3) Greater clustering (CF_1) narrows DDC’s advantage. For example, *temp_bin* and *year* have similar skew and cardinality, but CF_1 increases from 1.1 to 11.1, improving their relative performance. On *temp_bin*, the CRoaring, WAH, and EWAH COMP latency ratios are 3.05×, 10.94×, and 12.42×, respectively. On the more clustered *year*, these ratios decrease to 1.27×, 2.98×, and 2.69×.

IMDb, the real-world dataset used in the Join Order Benchmark (JOB) [7], contains data on movies, people, and production companies. Its attributes vary widely in cardinality, skewness, and CF_1 .

DDC Has the Smallest Aggregate Footprint. As shown in Table 5, DDC has the smallest total footprint (176.6MB). Column cardinality and skewness do not change this trend. For the two extremely clustered columns ($CF_1 = 1.3M$ and $2.8M$), WAH, EWAH, and CRoaring are smaller, but their absolute savings are small. For example, on *cast_info.role_id* ($CF_1 = 2.8M$), DDC is largest but uses less than 12KB more than any baseline. On the lightly clustered *cast_info.person_id* ($CF_1 = 7$), DDC is smallest, saving 30.2–93.8MB over the three baselines.

DDC Provides Fast Logical Bitwise Operations. We evaluate logical bitwise operations on bitmaps compressed by each scheme for the ten indexed IMDb attributes. The results follow the Berkeley Earth trends. The geometric-mean latency ratios of CRoaring, WAH, and EWAH relative to DDC are 23.6, 3.4, and 1.7, respectively. For brevity, we report detailed results in DDC’s appendix [1].

Conclusion. Real-world evaluations show that DDC is generally more space-efficient and faster than the baselines. DDC remains robust to column skewness and cardinality, consistent with the microbenchmark results in R3.W1. Highly clustered columns can

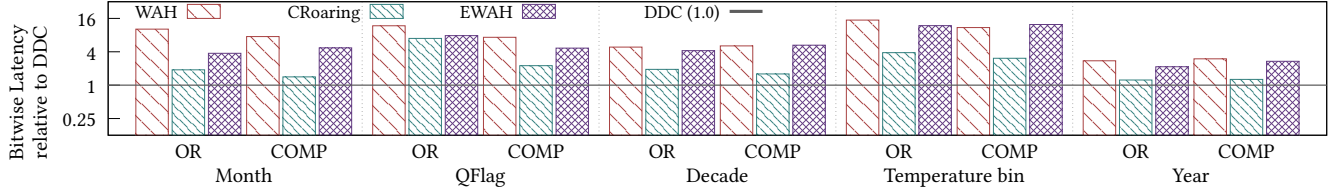


Figure 10: OR and COMP latency on five Berkeley Earth columns, normalized to DDC (lower is better). DDC is fastest in all ten cases, although its advantage narrows on more clustered columns.

Table 5: Characteristics and bitmap sizes for IMDb attributes (MB unless noted). DDC has the smallest total footprint (176.6MB) and uses the least space for every attribute with $CF_1 \leq 35$.

Bitmap	N	Card.	Density	CF_1	G	DDC	WAH		EWAH		CRoaring	
						Size	Size	Inc.	Size	Inc.	Size	Inc.
<i>movie_companies.company_type_id</i>	2.6M	2	0.5	1.3M	0.012	755B	40B	−95%	56B	−93%	590B	−22%
<i>title.kind_id</i>	2.5M	6	0.2	4	0.619	0.5	0.8	+62%	0.9	+65%	0.8	+50%
<i>cast_info.role_id</i>	36.2M	11	0.09	2.8M	0.547	12,107B	308B	−97%	432B	−96%	8,035B	−34%
<i>person_info.info_type_id</i>	3.0M	22	0.05	2	0.543	2.4	4.9	+1.1×	5.9	+1.5×	3.7	+56%
<i>movie_info.info_type_id</i>	14.8M	71	0.01	35	0.837	0.6	1.4	+1.3×	1.5	+1.4×	0.9	+54%
<i>title.production_year</i>	2.5M	132	0.007	1	0.696	3.9	10.5	+1.7×	14.7	+2.8×	4.9	+26%
<i>company_name.country_code</i>	235.0K	215	0.004	1	0.919	0.2	0.6	+1.5×	0.8	+2.5×	0.3	+20%
<i>movie_keyword.movie_id</i>	4.5M	476K	2×10^{-6}	9	0.688	2.5	10.0	+3.0×	11.9	+3.8×	6.6	+1.6×
<i>cast_info.movie_id</i>	36.2M	2.3M	4×10^{-7}	1	0.529	141.2	311.3	+1.2×	584.6	+3.1×	156.6	+11%
<i>cast_info.person_id</i>	36.2M	4.1M	2×10^{-7}	7	0.805	25.3	93.0	+2.7×	119.1	+3.7×	55.5	+1.2×
Total	—	—	—	—	—	176.6	432.4	+1.4×	739.3	+3.2×	229.2	+30%

favor WAH and CRoaring, but DDC’s gains on most uniform or lightly clustered columns outweigh these exceptions.

REFERENCES

- [1] 2026. DDC: Decoupling Data and Control for Bitmap Compression (Appendix). <https://junchangwang.github.io/doc/DDC-Appendix.pdf>. [Online; accessed 1-August-2026].
- [2] Manos Athanassoulis, Zheng Yan, and Stratos Idreos. 2016. UpBit: Scalable In-Memory Updatable Bitmap Indexing. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 1319–1332. <https://dl.acm.org/citation.cfm?id=2915964>
- [3] Berkeley. 2016. Berkeley Earth Data. <http://berkeleyearth.org/data/>. [Online; accessed 1-May-2024].
- [4] Guy Jacobson. 1989. Space-Efficient Static Trees and Graphs. In *Proceedings of the 30th Annual Symposium on Foundations of Computer Science (FOCS)*. 549–554. doi:10.1109/SFCS.1989.63533
- [5] Michael S. Kester, Manos Athanassoulis, and Stratos Idreos. 2017. Access Path Selection in Main-Memory Optimized Data Systems: Should I Scan or Should I Probe?. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 715–730. <http://dl.acm.org/citation.cfm?doid=3035918.3064049>
- [6] Harald Lang, Alexander Beischl, Viktor Leis, Peter A Boncz, Thomas Neumann, and Alfons Kemper. 2020. Tree-Encoded Bitmaps. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. 937–967. <https://doi.org/10.1145/3318464.3380588>
- [7] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter A Boncz, Alfons Kemper, and Thomas Neumann. 2015. How Good Are Query Optimizers, Really? *Proceedings of the VLDB Endowment* 9, 3 (2015), 204–215. <http://www.vldb.org/pvldb/vol9/p204-leis.pdf>
- [8] Daniel Lemire and Owen Kaser. 2011. Reordering columns for smaller indexes. *Information Sciences (ISCI)* 181, 12 (2011), 2550–2570. <http://dx.doi.org/10.1016/j.ins.2011.02.002>
- [9] Daniel Lemire, Owen Kaser, and Kamel Aouiche. 2010. Sorting improves word-aligned bitmap indexes. *Data & Knowledge Engineering (DKE)* 69, 1 (2010), 3–28. <http://dx.doi.org/10.1016/j.datak.2009.08.006>
- [10] Mikolaj Morzy, Tadeusz Morzy, Alexandros Nanopoulos, and Yannis Manolopoulos. 2003. Hierarchical Bitmap Index: An Efficient and Scalable Indexing Technique for Set-Valued Attributes. In *ADBIS*, Vol. 2798. Springer, 236–252. https://doi.org/10.1007/978-3-540-39403-7_19
- [11] Ali Pinar, Tao Tao, and Hakan Ferhatosmanoglu. 2005. Compressing Bitmap Indices by Data Reorganization. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 310–321. <http://dx.doi.org/10.1109/ICDE.2005.35>
- [12] Liwen Sun, Michael J. Franklin, Jiannan Wang, and Eugene Wu. 2016. Skipping-oriented Partitioning for Columnar Layouts. *Proceedings of the VLDB Endowment* 10, 4 (2016), 421–432. <http://www.vldb.org/pvldb/vol10/p421-sun.pdf>
- [13] Junchang Wang and Manos Athanassoulis. 2024. CUBIT: Concurrent Updatable Bitmap Indexing. *Proceedings of the VLDB Endowment* 18, 2 (2024), 399–412. <https://www.vldb.org/pvldb/vol18/p399-athanassoulis.pdf>
- [14] Kesheng Wu, Ekow J. Otoo, and Arie Shoshani. 2006. Optimizing Bitmap Indices with Efficient Compression. *ACM Transactions on Database Systems (TODS)* 31, 1 (2006), 1–38. <http://doi.acm.org/10.1145/1132863.1132864>